

1. Что такое монолитная архитектура?	4
2. Какие основные преимущества монолитной архитектуры?	4
3. В каких случаях предпочтительно использовать монолитную архитектуру?	4
1. Какие недостатки монолитной архитектуры могут повлиять на процесс разработки?	6
2. Как монолитная архитектура влияет на масштабирование приложения?	6
3. Какие стратегии можно использовать для разделения монолитной архитектуры на более мелкие компоненты?	6
1. Какие подходы к мониторингу и управлению производительностью наиболее эффективны для монолитных архитектур?	13
2. Какие технологии и инструменты лучше всего подходят для рефакторинга монолитной архитектуры?	13
3. Какие основные вызовы и проблемы связаны с миграцией с монолитной архитектуры на микросервисную?	13
1. Как микросервисная архитектура влияет на процесс разработки и сопровождения ПО?	17
2. Какие технологии и инструменты обычно используются при реализации микросервисной архитектуры?	17
3. В чем заключается принцип разделения ответственности в микросервисной архитектуре?	17
1. Какие подходы к масштабированию предполагает микросервисная архитектура и какие проблемы могут возникнуть при этом?	21
2. Как реализуется управление транзакциями в распределенной системе с микросервисной архитектурой?	21
3. Опишите, как реализуется обработка ошибок и обеспечение надежности в микросервисных системах.....	21
1. Что обозначают аббревиатуры CAP и PACELC в контексте распределенных систем?	24
2. Какой компонент теоремы CAP невозможно достичь одновременно с двумя другими?	24
3. Опишите основное отличие между теоремой CAP и теоремой PACELC.	24

1. Как влияет разделение (Partitioning) на доступность (Availability) и согласованность (Consistency) в теореме CAP?	26
2. На какие две части делится выбор в теореме PACELC и что они означают?	26
3. Какие типы распределенных систем лучше всего соответствуют каждому из трех принципов CAP (C, A, P)?	26
1. Объясните, как теорема PACELC расширяет и дополняет теорему CAP.	29
2. Какие компромиссы следует рассмотреть при проектировании системы с высокой доступностью в условиях разделения сети, согласно PACELC?	29
3. В каких сценариях использования распределенных систем предпочтение согласованности перед доступностью (и наоборот) может быть оправдано, исходя из теорем PACELC и CAP?	29
1. Что такое распределенная транзакция?	32
2. Какие основные проблемы связаны с реализацией распределенных транзакций в микросервисной архитектуре?	32
3. В чем различие между локальной и распределенной транзакциями?	32
1. Какие существуют подходы к управлению распределенными транзакциями в микросервисных архитектурах?	35
2. Что такое паттерн Saga и как он используется для обработки распределенных транзакций?	35
3. Какие проблемы решает применение паттерна Two-Phase Commit (2PC) в распределенных транзакциях?	35
1. Какие недостатки имеет паттерн Two-Phase Commit (2PC) и какие альтернативы могут быть рассмотрены?	39
2. Как реализовать обработку распределенных транзакций без блокировки ресурсов в микросервисной архитектуре?	39
3. Объясните принцип работы и преимущества паттерна Eventual Consistency в контексте распределенных транзакций.	39
1. Какие признаки указывают на то, что пора разделять монолит на микросервисы?	43
2. Какие риски несет разделение монолитного приложения на микросервисы?	43
3. Какие преимущества микросервисной архитектуры можно использовать для обоснования ее внедрения вместо монолита?	43

1. Какие стратегии разделения монолитного приложения на микросервисы вы знаете?	45
2. Какие технические и организационные аспекты необходимо учитывать при переходе от монолита к микросервисам?	45
3. Какие паттерны проектирования микросервисов помогут минимизировать проблемы с зависимостями и коммуникацией между сервисами?	45
1. Что такое многоуровневая архитектура приложений?	48
2. В чем заключается основная идея паттерна MVC (Model-View-Controller)?	48
3. Какие преимущества дает использование микросервисной архитектуры перед монолитной?	48
1. Какие основные принципы SOLID применимы к архитектуре приложений?	50
2. Как реализовать обмен данными между микросервисами?	50
1. Опишите процесс проектирования системы с использованием CQRS и Event Sourcing.	52
2. Какие вызовы и проблемы связаны с переходом от монолитной архитектуры к микросервисной?	52
3. В чем заключается подход Идемпотентности API и как он влияет на архитектуру системы?	52
1. Что такое DDD (Domain-Driven Design) и для чего он используется?	54
2. Какие основные компоненты входят в концепцию DDD?	54
3. Объясните понятие Ubiquitous Language в контексте DDD.	54
1. Какие преимущества предоставляет использование DDD при проектировании программного обеспечения?	56
2. В чем разница между Entity и Value Object в DDD?	56
3. Какова роль агрегатов в DDD и как они помогают управлять сложностью доменной модели?	56
1. Как DDD интегрируется с микросервисной архитектурой?	58
2. Обсудите важность и применение контекстных границ (Bounded Contexts) в DDD.	58
3. Какие стратегии вы бы предложили для реализации DDD в существующей системе с монолитной архитектурой?	58

1. Что такое монолитная архитектура?
2. Какие основные преимущества монолитной архитектуры?
3. В каких случаях предпочтительно использовать монолитную архитектуру?

1. Что такое монолитная архитектура

Монолитная архитектура — это подход к построению приложения, при котором **всё приложение представляет собой единое целое**:

- один кодовый базис,
- один артефакт сборки (JAR/WAR),
- один процесс выполнения,
- общая база данных (как правило).

Все модули (UI, бизнес-логика, доступ к данным, интеграции) **жёстко связаны** и разворачиваются **одновременно**.

✦ Пример: Spring Boot-приложение, которое содержит контроллеры, сервисы, репозитории и деплоится как один JAR.

2. Основные преимущества монолитной архитектуры

✓ Простота разработки

- Проще начать проект с нуля
- Нет межсервисных контрактов, сетевых вызовов, service discovery
- Удобно дебажить: один процесс, один стек

✓ Простота развертывания

- Один артефакт → один деплой
- Не нужны Kubernetes, API Gateway, Config Server и т. д.

✓ Производительность

- Вызовы между модулями — **внутрипроцессные**, а не по сети
- Меньше накладных расходов (latency, сериализация, retries)

✓ Простота тестирования

- Интеграционные тесты легче писать
- Меньше моков и контрактных тестов

✓ Низкий операционный оверхед

- Проще мониторинг, логирование, конфигурация
- Меньше инфраструктуры и DevOps-сложности

3. Когда предпочтительно использовать монолитную архитектуру

Монолит предпочтителен, если:

Небольшой или средний проект

- MVP
- Стартап
- Внутренние корпоративные системы

Небольшая команда

- 1–5 разработчиков
- Нет выделенной DevOps-команды

Бизнес-логика ещё нестабильна

- Часто меняются требования
- Архитектура активно эволюционирует

Нет высоких требований к масштабированию

- Низкая или средняя нагрузка
- Масштабирование возможно целиком (vertical / simple horizontal)

Важна скорость разработки

- Time-to-market критичен
- Нужно быстро выпускать фичи

1. Какие недостатки монолитной архитектуры могут повлиять на процесс разработки?
2. Как монолитная архитектура влияет на масштабирование приложения?
3. Какие стратегии можно использовать для разделения монолитной архитектуры на более мелкие компоненты?

1. Недостатки монолитной архитектуры, влияющие на процесс разработки

✗ Сильная связность компонентов

- Модули тесно зависят друг от друга
- Изменение в одной части может сломать другую
- Растёт риск регрессий

👉 В результате:

- сложнее вносить изменения,
- требуется больше регрессионного тестирования.

✗ Усложнение поддержки при росте проекта

- Кодовая база разрастается
- Ухудшается читаемость и навигация по проекту
- Порог входа для новых разработчиков повышается

👉 Команда начинает «бояться» изменений.

✗ Ограниченная параллельная разработка

- Несколько команд работают в одном репозитории
- Частые конфликты при merge
- Сложнее независимо развивать части системы

✗ Единый цикл релиза

- Любое изменение требует пересборки и перезапуска всего приложения
- Нельзя задеплоить один модуль отдельно

👉 Даже мелкий баг фиксируется «тяжёлым» релизом.

✗ Ограничения по технологиям

- Трудно использовать разные технологии/языки для разных частей
- Архитектурные решения «застывают» надолго

2. Влияние монолитной архитектуры на масштабирование

Масштабирование только целиком

- Нельзя масштабировать отдельный функционал
- При высокой нагрузке на один модуль приходится масштабировать всё приложение

Пример:

Если нагружен только модуль отчетов, всё приложение масштабируется ради него.

Неэффективное использование ресурсов

- Лишние инстансы потребляют память и CPU
- Увеличиваются затраты на инфраструктуру

Ограничения по отказоустойчивости

- Сбой в одном компоненте может «уронить» всё приложение
- Нет изоляции ошибок

3. Стратегии разделения монолита на более мелкие компоненты

0 Главное, что нужно понять заранее

Монолит НЕ делят сразу на микросервисы

Почти всегда сначала его делят на логические компоненты (модули), и только потом — при необходимости — выносят их в отдельные сервисы.

То есть путь обычно такой:

Монолит
↓
Модульный монолит
↓
Выделенные сервисы (микросервисы)

1 Стратегия №1 — Логическое разделение по бизнес-доменам (DDD)

👉 Самая правильная и долгосрочная стратегия

Идея

Разделяем монолит **по бизнес-смыслу**, а не по техническим слоям.

❌ Плохо:

```
controllers
services
repositories
```

✅ Хорошо:

```
order
payment
delivery
user
```

Как делать на практике

1. Выписываешь **бизнес-функции**:
 - Заказы
 - Оплата
 - Пользователи
 - Уведомления
2. Для каждой функции:
 - свои сущности
 - свои сервисы
 - свои репозитории

Пример структуры:

```
monolith
├── order
│   ├── Order
│   ├── OrderService
│   └── OrderRepository
├── payment
│   ├── Payment
│   ├── PaymentService
│   └── PaymentRepository
└── user
    ├── User
    ├── UserService
    └── UserRepository
```

❖ **Правило:** код из `order` не лезет напрямую в `payment`.

2 Стратегия №2 — Модульный монолит (самый важный этап)

👉 Это «золотой стандарт» перед микросервисами

Что это такое

Приложение всё ещё одно, но:

- модули жёстко изолированы
 - общение только через **публичные интерфейсы**
-

Практика (Java / Spring)

```
order-module  
payment-module  
user-module
```

Каждый модуль:

- имеет **public API**
- скрывает внутренние классы

```
// order-api  
public interface OrderService {  
    Order createOrder(CreateOrderCommand cmd);  
}  
// payment-module  
@Autowired  
private OrderService orderService; // через интерфейс
```

✗ Нельзя:

- напрямую ходить в чужие репозитории
 - шарить Entity между модулями
-

Что даёт

- Можно **вынести модуль в микросервис без переписывания логики**
 - Чёткий контракт между частями системы
-

3 Стратегия №3 — Выделение по данным (Database-first)

👉 Используется, если код хаотичный, но БД более-менее понятна

Идея

Разделять по **таблицам и данным**, а не по коду.

Пример:

- `orders, order_items` → Order component
- `payments, transactions` → Payment component

Шаги

1. Определяешь владельца каждой таблицы
2. Запрещаешь прямой доступ к «чужим» таблицам
3. Доступ только через сервис

✦ Это подготовка к правилу микросервисов:

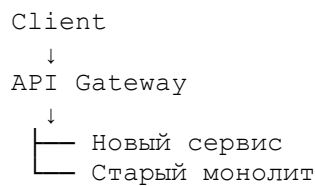
«Один сервис — одна база данных»

4 Стратегия №4 — Strangler Fig Pattern (удушающее дерево)

👉 Самая безопасная стратегия

Идея

Не ломать монолит, а постепенно «обрастать» новыми компонентами.



Как это работает

1. Новый функционал — **только вне монолита**
2. Старые части постепенно переписываются
3. Монолит со временем «усыхает»

📌 Очень популярно в enterprise-проектах.

5 Стратегия №5 — Выделение по нагрузке (Performance-driven)

👉 Когда система уже под нагрузкой

Пример:

- 90% нагрузки — чтение каталога
 - Выносим каталог в отдельный сервис
-

Типичные кандидаты:

- Поиск
- Каталоги
- Отчёты
- Нотификации

6 Стратегия №6 — Событийное разделение (Event-driven)

👉 Для сложных бизнес-процессов

Вместо:

```
OrderService → PaymentService → DeliveryService
```

Используем:

```
OrderCreated event  
PaymentProcessed event
```

Каждый компонент:

- реагирует на события
- не знает, кто их отправил

✦ Отлично сочетается с:

- DDD
- Saga
- Eventual Consistency

7 В каком порядке лучше применять стратегии

🔥 Рекомендованный порядок:

- 1 DDD + Bounded Context
- 2 Модульный монолит
- 3 Изоляция данных
- 4 Strangler Pattern
- 5 Выделение сервисов
- 6 Event-driven

1. Какие подходы к мониторингу и управлению производительностью наиболее эффективны для монолитных архитектур?
2. Какие технологии и инструменты лучше всего подходят для рефакторинга монолитной архитектуры?
3. Какие основные вызовы и проблемы связаны с миграцией с монолитной архитектуры на микросервисную?

1. Мониторинг и управление производительностью в монолитной архитектуре

Для монолита ключевая задача — видимость происходящего внутри одного процесса.

✓ Логирование

Цель: понять поведение приложения и причины ошибок.

Лучшие практики:

- структурированные логи (JSON),
- корреляционные ID (requestId),
- логирование времени выполнения операций.

Инструменты:

- Logback / Log4j2
- ELK (Elasticsearch + Logstash + Kibana)
- Loki + Grafana

✓ Метрики

Цель: количественная оценка состояния приложения.

Ключевые метрики:

- latency (p95 / p99),
- throughput,
- error rate,
- использование CPU / памяти / GC.

Инструменты:

- Micrometer + Prometheus
- Grafana
- Spring Boot Actuator

✓ Профилирование

Цель: найти узкие места (CPU, память, блокировки).

Инструменты:

- JVisualVM
- JProfiler
- Java Flight Recorder (JFR)
- JMC (Java Mission Control)

✦ Особенно эффективно для монолита, т.к. всё в одном JVM-процессе.

✓ Трассировка (внутрипроцессная)

Хотя монолит — один сервис, **distributed tracing** всё равно полезен:

- анализ долгих цепочек вызовов,
- выявление медленных методов.

Инструменты:

- OpenTelemetry (внутри JVM)
- Zipkin / Jaeger

2. Технологии и инструменты для рефакторинга монолитной архитектуры

✓ Статический анализ кода

Помогает выявить плохие зависимости и технический долг.

Инструменты:

- SonarQube
- ArchUnit (архитектурные тесты)
- Checkstyle / PMD

✓ Модульность и архитектурные ограничения

Цель: превратить «грязный монолит» в модульный.

Подходы и инструменты:

- Java Platform Module System (JPMS)
- Spring Modulith
- ArchUnit для контроля границ модулей

✓ Рефакторинг на уровне домена

DDD-подход:

- выделение bounded context,
- устранение shared mutable state,
- явные контракты между модулями.

Инструменты:

- Event Storming (как процесс, не инструмент)
- C4 Model для визуализации архитектуры

✓ Инфраструктурные инструменты

- Feature toggles (Unleash, Togglz)
- Message brokers (Kafka, RabbitMQ) — сначала внутри монолита
- API Gateway (на этапе миграции)

Основные вызовы и проблемы миграции с монолита на микросервисы

✗ Ошибочное разделение домена

- Сервисы делятся «по таблицам», а не по бизнесу
- Возникает tight coupling между сервисами

👉 Самая частая причина провала миграции.

✗ Работа с данными

- Распределённые транзакции
- Потеря ACID
- Необходимость eventual consistency

✦ Требуется:

- Saga / Outbox паттернов
- Event-driven подхода

✗ Сетевые проблемы

- latency
- timeouts
- retries
- partial failures

👉 В монолите этого просто нет.

✗ Рост операционной сложности

- CI/CD
- мониторинг множества сервисов
- конфигурации
- service discovery

✦ Без сильного DevOps — боль.

✗ Тестирование

- сложнее интеграционные тесты
 - необходимость контрактных тестов
 - нестабильность тестовых окружений
-

✗ Организационные вызовы

- команды не готовы к автономности
- отсутствие ownership
- неправильные границы ответственности

1. Как микросервисная архитектура влияет на процесс разработки и сопровождения ПО?
2. Какие технологии и инструменты обычно используются при реализации микросервисной архитектуры?
3. В чем заключается принцип разделения ответственности в микросервисной архитектуре?

1. Влияние микросервисной архитектуры на разработку и сопровождение ПО

✓ Положительное влияние

◆ Независимая разработка и деплой

- Каждый сервис развивается и релизится отдельно
- Меньше блокировок между командами
- Быстрее time-to-market

◆ Масштабируемость команд

- Команды автономны
- Чёткое ownership сервиса
- Параллельная разработка без конфликтов

◆ Технологическая гибкость

- Разные сервисы могут использовать разные технологии
- Проще экспериментировать и обновлять стек

✗ Негативное влияние

▼ Рост сложности системы

- Распределённая система по определению сложнее
- Появляются сетевые ошибки, таймауты, частичные отказы

▼ Усложнение отладки

- Нет единого стека вызовов
- Нужны distributed tracing и correlation ID

▼ Повышенные требования к инфраструктуре

- CI/CD
- мониторинг
- логирование
- управление конфигурацией

✦ Итог: скорость разработки растёт, но цена сопровождения выше.

2. Технологии и инструменты микросервисной архитектуры

◆ Коммуникация между сервисами

- REST (Spring Web, OpenAPI)
 - gRPC
 - Асинхронные сообщения: Kafka, RabbitMQ
-

◆ Инфраструктура

- Docker
 - Kubernetes
 - Helm
-

◆ Service Discovery и маршрутизация

- Kubernetes DNS
 - Eureka / Consul
 - API Gateway (Spring Cloud Gateway, Kong)
-

◆ Конфигурация и устойчивость

- Spring Cloud Config
 - Resilience4j (retry, circuit breaker)
 - Rate limiting
-

◆ Наблюдаемость (Observability)

- Prometheus + Grafana (метрики)
 - ELK / Loki (логи)
 - Jaeger / Zipkin (трейсинг)
 - OpenTelemetry
-

◆ Безопасность

- OAuth2 / OpenID Connect
 - Keycloak
 - mTLS
-

3. Принцип разделения ответственности в микросервисной архитектуре

🔑 Суть принципа

Один микросервис = одна бизнес-ответственность.

Микросервис:

- отвечает за **один бизнес-контекст**,
 - владеет **своими данными**,
 - имеет **чёткий контракт (API)**,
 - может развиваться независимо.
-

📌 Что это означает на практике

✅ Изоляция данных

- У сервиса **своя БД или схема**
 - Нет прямого доступа к данным других сервисов
-

✅ Явные контракты

- Общение только через API или события
 - Никаких shared libraries с бизнес-логикой
-

 Независимость жизненного цикла

- Свой CI/CD
- Свой релизы
- Своя стратегия масштабирования

 Частые нарушения принципа

- Общая база данных на несколько сервисов
- Сервисы «по слоям» (user-service, repository-service)
- Сильная синхронная связность

1. Какие подходы к масштабированию предполагает микросервисная архитектура и какие проблемы могут возникнуть при этом?
2. Как реализуется управление транзакциями в распределенной системе с микросервисной архитектурой?
3. Опишите, как реализуется обработка ошибок и обеспечение надежности в микросервисных системах.

1. Масштабирование в микросервисной архитектуре и возникающие проблемы

Подходы к масштабированию

✓ Горизонтальное масштабирование (основное)

- Каждый микросервис масштабируется **независимо**
- Добавление инстансов под нагрузкой
- Обычно реализуется через Kubernetes (HPA)

Плюсы:

- эффективное использование ресурсов
 - масштабируется только «узкое место»
-

✓ Вертикальное масштабирование

- Увеличение CPU / RAM конкретного сервиса
 - Используется реже, как временная мера
-

✓ Масштабирование по типу нагрузки

- Read/Write separation
- Кэширование (Redis)
- Асинхронная обработка через очереди

Проблемы масштабирования

✗ Сетевая нестабильность

- latency
- packet loss
- таймауты

👉 Нужно проектировать систему как **ненадёжную по умолчанию**.

✗ Состояние (state)

- Stateful-сервисы плохо масштабируются
- Требуется:
 - вынос состояния во внешние хранилища,
 - sticky sessions или их устранение.

✗ Каскадные отказы

- Перегруженный сервис может «потянуть» за собой другие
- Усиливается при синхронных вызовах

2. Управление транзакциями в микросервисной архитектуре

✗ Почему нельзя классические ACID-транзакции

- Нет общей БД
- ХА / 2PC плохо масштабируются и нестабильны
- Высокая связность и latency

Основные подходы

✓ Saga Pattern (основной)

Транзакция = цепочка локальных транзакций

Варианты:

- Orchestration (централизованный координатор)
- Choreography (реакция на события)

✦ Используются **компенсирующие действия**, а не rollback.

✓ Eventual Consistency

- Данные становятся согласованными **со временем**
- Система проектируется с учётом временной неконсистентности

✓ Outbox Pattern

- Событие и бизнес-операция сохраняются в одной локальной транзакции
- Гарантирует доставку событий

✗ Антипаттерны

- Shared database
- Distributed transactions через 2PC

3. Обработка ошибок и обеспечение надёжности

Основной принцип

Любой сетевой вызов может упасть. Всегда.

Паттерны надёжности

✔ Circuit Breaker

- Прерывает вызовы к «падающему» сервису
- Даёт системе восстановиться

Инструменты:

- Resilience4j

✔ Retry + Backoff

- Повтор запросов при временных ошибках
- Обязательно:
 - лимиты,
 - экспоненциальная задержка,
 - idempotency

✔ Timeout

- Явные таймауты на все вызовы
- Без таймаута — потенциальный deadlock

✔ Bulkhead

- Изоляция ресурсов
- Ошибка в одном компоненте не валит всё приложение

Асинхронная обработка ошибок

- Dead Letter Queue (DLQ)
- Повторная обработка сообщений
- Ручная компенсация

Наблюдаемость

- Correlation ID
- Distributed tracing
- Centralized logging

1. Что обозначают аббревиатуры CAP и PACELC в контексте распределенных систем?
2. Какой компонент теоремы CAP невозможно достичь одновременно с двумя другими?
3. Опишите основное отличие между теоремой CAP и теоремой PACELC.

1. Что обозначают CAP и PACELC

◆ CAP-теорема

CAP — это ограничение распределённых систем, которое утверждает:

В условиях сетевого разделения (Partition) распределённая система может гарантировать **либо согласованность (Consistency), либо доступность (Availability)**, но не обе одновременно.

Расшифровка:

- **C (Consistency)** — все узлы видят одинаковые данные в один и тот же момент времени
- **A (Availability)** — каждый запрос получает ответ (не обязательно актуальный)
- **P (Partition tolerance)** — система продолжает работать при сетевых разделениях

◆ PACELC-теорема

PACELC расширяет CAP и говорит:

Если есть разделение сети (**P**) — выбор между **A** и **C**,
иначе (**E — Else**) — выбор между **L (Latency)** и **C (Consistency)**.

Расшифровка:

- **P** — Partition
- **A** — Availability
- **C** — Consistency
- **E** — Else (когда разделения нет)
- **L** — Latency

2. Какой компонент CAP невозможно достичь одновременно с двумя другими

👉 Невозможно одновременно обеспечить Consistency и Availability при наличии Partition tolerance.

То есть:

- если система должна быть устойчивой к partition (P),
- то при partition она выбирает:
 - либо C + P (жертвуя доступностью),
 - либо A + P (жертвуя строгой согласованностью).

✚ P в реальных распределённых системах не обсуждается — он обязателен.

3. Основное отличие CAP и PACELC

◆ CAP

- Рассматривает только ситуацию сетевого разделения
- Ничего не говорит о поведении системы в нормальных условиях

◆ PACELC

- Учитывает реальный мир
- Описывает компромисс даже без partition:
 - низкая задержка vs строгая согласованность

✚ Пример:

- синхронная репликация → выше consistency, выше latency
- асинхронная репликация → ниже latency, weaker consistency

1. Как влияет разделение (Partitioning) на доступность (Availability) и согласованность (Consistency) в теореме CAP?
2. На какие две части делится выбор в теореме PACELC и что они означают?
3. Какие типы распределенных систем лучше всего соответствуют каждому из трех принципов CAP (C, A, P)?

1. Влияние Partitioning на Availability и Consistency (CAP)

◆ Что такое Partitioning (P)

Partitioning — это ситуация, когда сеть делится на изолированные части, и узлы не могут обмениваться сообщениями.

✚ В реальных распределённых системах:

- сетевые разделения **неизбежны**,
- поэтому **P считается обязательным**.

◆ Влияние на выбор A или C

При возникновении partition система **обязана выбрать**:

✅ Consistency (CP)

- система **отказывается отвечать** на часть запросов,
- ждёт восстановления связи,
- сохраняет единое корректное состояние данных.

👉 **Availability страдает** (часть запросов получает ошибки или таймауты).

✅ Availability (AP)

- система **продолжает отвечать** на все запросы,
- но данные могут быть **временно неконсистентными**.

👉 **Consistency страдает**, но сервис остаётся доступным.

✚ **Одновременно обеспечить C и A при наличии P невозможно.**

2. Две части выбора в теореме PACELC

PACELC делит выбор системы на две ситуации:

◆ $P \rightarrow A$ или C

Если происходит сетевое разделение (Partition)
выбираем между:

- Availability (A)
- Consistency (C)

◆ E (Else) $\rightarrow L$ или C

Если сетевого разделения нет
выбираем между:

- Latency (L) — меньшая задержка
- Consistency (C) — строгая согласованность

✦ PACELC подчёркивает:

- компромиссы есть **всегда**, а не только при авариях.

3. Типы распределённых систем для принципов CAP

⚠ Важно:

Нет “чистых” C, A или P систем.

Обычно говорят о CP и AP системах (P предполагается).

◆ CP-системы (Consistency + Partition tolerance)

Характеристики:

- жертвуют доступностью при partition
- строгая согласованность данных

Подходят для:

- финансовых систем
- систем учёта
- критичных транзакций

Примеры:

- HBase
- MongoDB (в режиме majority write concern)
- Zookeeper

◆ AP-системы (Availability + Partition tolerance)

Характеристики:

- всегда отвечают на запросы
- допускают eventual consistency

Подходят для:

- социальные сети
- каталоги товаров
- аналитические системы

Примеры:

- Cassandra
 - DynamoDB
 - CouchDB
-

◆ CA-системы (Consistency + Availability)

📌 Теоретически возможны только без partition
(то есть не являются реально распределёнными).

Примеры:

- одиночная БД
- кластер в одной машине / без сетевых отказов

1. Объясните, как теорема PACELC расширяет и дополняет теорему CAP.
2. Какие компромиссы следует рассмотреть при проектировании системы с высокой доступностью в условиях разделения сети, согласно PACELC?
3. В каких сценариях использования распределенных систем предпочтение согласованности перед доступностью (и наоборот) может быть оправдано, исходя из теорем PACELC и CAP?

1. Как PACELC расширяет и дополняет CAP

◆ Ограниченность CAP

CAP отвечает только на один вопрос:

Что делать системе при сетевом разделении (Partition)?

И утверждает:

- при **P** выбираем **C** или **A**.

! Но CAP ничего не говорит, как система ведёт себя в нормальных условиях, когда сеть стабильна.

◆ Что добавляет PACELC

PACELC говорит:

P → **A** или **C**
Else → **L** или **C**

То есть:

- при **partition** — тот же выбор, что и в CAP,
- без **partition** — компромисс между:
 - **Latency (L)** — быстрые ответы,
 - **Consistency (C)** — строгая согласованность.

✦ PACELC подчёркивает:

- компромиссы есть **всегда**, а не только в аварийных ситуациях,
- выбор consistency почти всегда увеличивает latency.

2. Компромиссы для систем с высокой доступностью при partition (PACELC)

Если цель — **высокая доступность**, система выбирает **AP** при **P**.

◆ Основные компромиссы

✗ Ослабленная согласованность

- возможны stale-данные
 - конфликты при записи
 - временные расхождения между репликами
-

✗ Сложность бизнес-логики

- система должна уметь:
 - разрешать конфликты,
 - обрабатывать дубликаты,
 - жить с eventual consistency.
-

✗ Компенсации вместо rollback

- нельзя откатить глобальную транзакцию
 - используются:
 - Saga,
 - компенсационные действия.
-

◆ Выбор при Else (нет partition)

Даже при стабильной сети нужно решить:

- **низкая latency** (асинхронная репликация),
 - или **строгая consistency** (синхронная репликация).
-

3. Когда оправдан выбор Consistency или Availability

◆ Предпочтение Consistency (CP)

Оправдано, когда:

- данные критичны
- ошибки дороже простоя

✚ Сценарии:

- банковские транзакции
- платёжные системы
- учёт остатков / инвентаря
- системы авторизации и прав доступа

👉 Лучше отказать в запросе, чем вернуть неверный результат.

◆ Предпочтение Availability (AP)

Оправдано, когда:

- непрерывность сервиса важнее точности
- допустимы временные расхождения

✚ Сценарии:

- социальные сети
- ленты новостей
- каталоги товаров
- системы рекомендаций

👉 Лучше показать немного устаревшие данные, чем не ответить вообще.

◆ Гибридный подход (часто на практике)

- разные части системы делают **разный выбор**
- read — AP
- write — CP
- разные SLA для разных операций

✚ Это признак зрелой архитектуры.

1. Что такое распределенная транзакция?
2. Какие основные проблемы связаны с реализацией распределенных транзакций в микросервисной архитектуре?
3. В чем различие между локальной и распределенной транзакциями?

1. Что такое распределённая транзакция

Распределённая транзакция — это транзакция, которая охватывает **несколько независимых ресурсов**:

- разные базы данных,
- разные микросервисы,
- разные узлы сети.

Её цель — обеспечить **атомарность и согласованность** изменений во всех участниках:

либо изменения применяются **везде**,
либо **нигде**.

✦ Классический пример:

- перевод денег между счетами в разных БД / сервисах.

2. Основные проблемы распределённых транзакций в микросервисной архитектуре

✗ Сетевая ненадёжность

- таймауты
- разрывы соединений
- частичные отказы

👉 Транзакция может «зависнуть» в неопределённом состоянии.

✗ Высокая задержка (latency)

- синхронная координация нескольких сервисов
- блокировки держатся дольше

✗ Проблемы масштабируемости

- протоколы вроде **2PC (Two-Phase Commit)** плохо масштабируются
- координатор становится узким местом

✗ Потеря доступности (CAP)

- при partition система вынуждена:
 - блокировать операции,
 - либо отказывать в обслуживании
-

✗ Сильная связность сервисов

- сервисы вынуждены знать о транзакционном контексте друг друга
 - нарушается автономность микросервисов
-

✗ Операционная сложность

- сложная отладка
 - трудно воспроизводимые ошибки
 - сложное восстановление после сбоев
-

✦ Поэтому классические распределённые ACID-транзакции считаются антипаттерном в микросервисах.

3. Различие между локальной и распределённой транзакциями

◆ Локальная транзакция

Характеристики:

- работает в рамках **одного ресурса**
- полностью поддерживает **ACID**
- управляется одной СУБД

✦ Пример:

- транзакция в одной БД внутри одного сервиса.

Плюсы:

- надёжность
- высокая производительность
- простота реализации

◆ Распределённая транзакция

Характеристики:

- затрагивает **несколько ресурсов**
- требует координации
- ACID достигается с большими ограничениями

✦ Пример:

- операция, затрагивающая несколько микросервисов.

Минусы:

- высокая latency
- низкая отказоустойчивость
- плохая масштабируемость

Короткая таблица сравнения

Критерий	Локальная	Распределённая
Кол-во ресурсов	1	>1
ACID	Полный	Ограниченный
Задержка	Низкая	Высокая
Масштабируемость	Хорошая	Плохая
Надёжность	Высокая	Низкая
Сложность	Низкая	Высокая

1. Какие существуют подходы к управлению распределенными транзакциями в микросервисных архитектурах?
2. Что такое паттерн Saga и как он используется для обработки распределенных транзакций?
3. Какие проблемы решает применение паттерна Two-Phase Commit (2PC) в распределенных транзакциях?

1. Подходы к управлению распределёнными транзакциями в микросервисах

В микросервисной архитектуре классические ACID-распределённые транзакции избегают, поэтому используются альтернативные подходы.

✓ 1. Saga Pattern (основной и рекомендуемый)

- Транзакция разбивается на **цепочку локальных транзакций**
- Каждая локальная транзакция имеет **компенсирующее действие**
- Система работает в режиме **eventual consistency**

✦ Используется **почти всегда** в микросервисах.

✓ 2. Eventual Consistency

- Система допускает временную несогласованность
 - Данные становятся согласованными со временем
 - Требуется продуманной бизнес-логики
-

✓ 3. Outbox + Inbox Patterns

- Гарантированная доставка событий
 - Избежание потери сообщений
 - Устранение проблемы «двойной записи» (DB + message broker)
-

⚠ 4. Two-Phase Commit (2PC)

- Используется редко
 - Подходит только для узких, контролируемых сценариев
 - Обычно **антипаттерн** в микросервисах
-

2. Паттерн Saga: что это и как используется

◆ Что такое Saga

Saga — это паттерн управления распределёнными транзакциями, при котором:

Глобальная транзакция = последовательность локальных транзакций

- компенсирующие операции вместо rollback.
-

◆ Варианты реализации Saga

✓ Choreography

- Сервисы обмениваются событиями
- Нет центрального координатора

Плюсы:

- слабая связность
- хорошая масштабируемость

Минусы:

- сложнее отслеживать поток
 - риск «event storm»
-

✓ Orchestration

- Есть центральный orchestrator
- Он управляет шагами Saga

Плюсы:

- проще контролировать процесс
- лучше наблюдаемость

Минусы:

- дополнительная точка отказа
 - риск «God-service»
-

◆ Пример (упрощённо)

1. Order Service → создаёт заказ
 2. Payment Service → списывает деньги
 3. Inventory Service → резервирует товар
- ✗ Ошибка на шаге 3 →
→ компенсация: возврат денег + отмена заказа
-

3. Какие проблемы решает Two-Phase Commit (2PC)

◆ Суть 2PC

2PC — протокол, обеспечивающий **атомарность распределённой транзакции**.

Фазы:

1. **Prepare** — все участники подтверждают готовность
 2. **Commit / Rollback** — координатор завершает транзакцию
-

◆ Какие проблемы решает 2PC

✓ Атомарность

- либо commit во всех системах,
 - либо rollback во всех
-

✓ Строгая согласованность

- нет временной неконсистентности
 - данные всегда согласованы
-

▼ Но за счёт чего

✗ Блокировки

- ресурсы держатся заблокированными до конца транзакции

✗ Плохая отказоустойчивость

- координатор — SPOF
- сбой → «подвисшие» транзакции

✗ Плохая масштабируемость

- рост latency
 - плохо работает при высокой нагрузке
-

📌 Поэтому:

- **2PC решает проблему атомарности,**
- **но создаёт критические проблемы доступности и масштабирования.**

1. Какие недостатки имеет паттерн Two-Phase Commit (2PC) и какие альтернативы могут быть рассмотрены?
2. Как реализовать обработку распределенных транзакций без блокировки ресурсов в микросервисной архитектуре?
3. Объясните принцип работы и преимущества паттерна Eventual Consistency в контексте распределенных транзакций.

1. Недостатки паттерна Two-Phase Commit (2PC) и альтернативы

◆ Основные недостатки 2PC

1. **Блокировки ресурсов**
 - Участники держат транзакции открытыми до завершения commit/rollback.
 - При медленных или зависших участниках ресурсы блокируются, что снижает throughput.
 2. **Проблемы отказоустойчивости**
 - Координатор 2PC — Single Point of Failure.
 - Сбой координатора или сети → «подвисшие транзакции».
 3. **Высокая задержка (latency)**
 - Все участники должны согласовать готовность перед commit.
 - Каждая транзакция синхронная → увеличивает время ответа.
 4. **Плохая масштабируемость**
 - При росте числа участников latency и риск ошибок растут.
 - Масштабирование распределённых транзакций становится сложным.
-

◆ Альтернативы 2PC

- **Saga Pattern**
 - Локальные транзакции с компенсационными действиями.
 - Eventual consistency вместо строгой атомарности.
 - **Eventual Consistency**
 - Данные могут быть временно несогласованными.
 - Подход с асинхронной репликацией и обработкой событий.
 - **Outbox / Inbox Pattern**
 - Гарантированная доставка сообщений и согласованность между БД и брокером.
-

2. Как реализовать распределённые транзакции без блокировки ресурсов

◆ Основные подходы

1. **Saga Pattern**
 - Каждая локальная транзакция коммитится сразу.
 - В случае ошибки выполняется **компенсирующее действие**.
 - Нет блокировок, т.к. шаги независимы.
2. **Асинхронная обработка через очереди**
 - Kafka, RabbitMQ, или другой брокер.
 - Сообщения доставляются надёжно, но обработка происходит позже.
3. **Outbox Pattern**
 - Локальная транзакция + запись события в отдельную таблицу (outbox).
 - Событие потом публикуется в брокер без блокировок основной операции.
4. **Idempotency и retry**
 - Повторные попытки операций безопасны.
 - Позволяет обрабатывать временные сбои без блокировки ресурсов.

3. Принцип работы и преимущества Eventual Consistency

◆ Принцип работы

- Система допускает временную несогласованность данных.
- Все изменения распространяются **асинхронно**.
- Со временем все реплики **сходятся к одинаковому состоянию**.

Пример:

1. Сервис А изменяет запись.
2. Событие отправляется в очередь.
3. Сервисы В и С обрабатывают событие позже.
4. Через некоторое время все сервисы имеют согласованные данные.

◆ Преимущества Eventual Consistency

1. **Высокая доступность**
 - Сервис продолжает работать даже при сетевых сбоях.
 - Нет блокировок на глобальные commit.
2. **Масштабируемость**
 - Асинхронные операции легко масштабируются горизонтально.
 - Нет узких мест, связанных с координатором транзакции.
3. **Отказоустойчивость**
 - Ошибки в одном сервисе не блокируют всю систему.
 - Можно реализовать повторные попытки и компенсации.
4. **Гибкость для микросервисов**
 - Сервисы остаются автономными.
 - Нет жесткой привязки к одной БД или одному протоколу.

1. Что такое монолитная архитектура?

2. В чем основное преимущество монолитной архитектуры перед микросервисной?

3. Почему монолитная архитектура может стать проблемой при масштабировании проекта?

1. Что такое монолитная архитектура

Монолитная архитектура — это структура приложения, в которой **весь код и функциональность собраны в одном едином приложении**:

- единый процесс/сервис, который выполняет всю бизнес-логику, работу с данными и пользовательский интерфейс,
- все модули сильно связаны между собой,
- деплой и запуск — одной сущности.

✦ Примеры: традиционные веб-приложения на Spring Boot без разделения на сервисы, старые ERP-системы.

2. Основное преимущество монолита перед микросервисной архитектурой

◆ Простота разработки и деплоя

- Один проект → один билд → один деплой, нет необходимости настраивать взаимодействие между сервисами.
- Меньше инфраструктурной сложности: не нужны брокеры сообщений, service discovery, распределённый мониторинг.

◆ Простота тестирования

- Интеграционные тесты проще, т.к. всё находится в одном процессе.

✦ Для небольших команд и проектов монолит быстрее внедрять и сопровождать.

3. Почему монолит может стать проблемой при масштабировании

◆ Ограниченная масштабируемость

- Невозможно масштабировать отдельные компоненты независимо.
- Любой рост нагрузки требует масштабирования всего приложения целиком.

◆ Рост сложности

- С ростом кода и функций появляется «spaghetti code».
- Новые разработчики труднее разбираются в архитектуре.

◆ Замедление разработки

- Любое изменение может затронуть весь монолит → увеличивается риск регрессий.
- CI/CD становится медленным: rebuild и redeploy всего приложения.

◆ Сложности в поддержке и обновлении

- Даже небольшие изменения требуют деплоя всего приложения.
- Модульность низкая → технический долг накапливается быстрее.

1. Какие признаки указывают на то, что пора разделять монолит на микросервисы?
2. Какие риски несет разделение монолитного приложения на микросервисы?
3. Какие преимущества микросервисной архитектуры можно использовать для обоснования ее внедрения вместо монолита?

1. Признаки того, что монолит пора разделять на микросервисы

◆ Рост сложности и технического долга

- Множество модулей в одном приложении трудно поддерживать.
- Сложно вносить изменения без риска поломать другие части.

◆ Замедление разработки и деплоя

- Частые конфликты между командами из-за общей кодовой базы.
- Весь монолит нужно пересобрать и деплоить даже для небольших изменений.

◆ Масштабируемость

- Одно узкое место (например, процесс обработки платежей) требует масштабирования всего приложения.

◆ Разные требования к технологиям

- Разные модули требуют разных технологий или БД.
- В монолите сложно использовать разные стеки для разных частей системы.

◆ Частые изменения отдельных частей

- Когда одни функции развиваются быстрее других, имеет смысл выделить их в отдельные сервисы.

2. Риски разделения монолита на микросервисы

✗ Рост операционной сложности

- Нужно настраивать **service discovery**, мониторинг, логирование, CI/CD для множества сервисов.

✗ Сетевая нестабильность

- В микросервисах добавляются **сетевые вызовы**, которые могут падать или быть медленными.

✗ Сложность управления данными

- Нет глобальных ACID-транзакций.
- Нужно проектировать eventual consistency, Saga или другие паттерны.

✗ Усложнение тестирования

- Интеграционные тесты становятся сложнее.
- Появляются межсервисные контракты, которые нужно контролировать.

✗ Организационные риски

- Команды должны быть готовы к автономности.
- Неправильное разделение контекста может привести к tight coupling между сервисами.

3. Преимущества микросервисной архитектуры для обоснования внедрения

✓ Независимые команды и деплой

- Команды работают автономно, можно деплоить сервисы отдельно.
- Ускоряет time-to-market.

✓ Масштабируемость отдельных компонентов

- Горизонтальное масштабирование только нужных сервисов, а не всего приложения.

✓ Технологическая гибкость

- Разные сервисы могут использовать разные языки программирования, фреймворки и базы данных.

✓ Устойчивость к сбоям

- Ошибка одного сервиса не ломает всю систему (с использованием circuit breaker, bulkhead и retry).

✓ Простота внедрения новых функций

- Новые сервисы можно добавить без сильного влияния на существующие части приложения.

✓ Наблюдаемость и мониторинг

- Легче отслеживать производительность и ошибки каждого сервиса.

1. Какие стратегии разделения монолитного приложения на микросервисы вы знаете?
2. Какие технические и организационные аспекты необходимо учитывать при переходе от монолита к микросервисам?
3. Какие паттерны проектирования микросервисов помогут минимизировать проблемы с зависимостями и коммуникацией между сервисами?

1. Стратегии разделения монолита на микросервисы

◆ Стратегия по бизнес-доменам (Domain-Driven Design, DDD)

- Разделение по **Bounded Contexts** — логическим областям бизнеса.
 - Каждый сервис отвечает за отдельный бизнес-контекст.
 - Пример: `Order Service`, `Payment Service`, `Inventory Service`.
-

◆ Стратегия по функциональности или модулям

- Разделяем по существующим модулям/подсистемам монолита.
 - Быстрый вариант, подходит для постепенного перехода.
-

◆ Стратегия по нагрузке и масштабируемости

- Выделяем узкие места, которые требуют горизонтального масштабирования.
 - Пример: сервис обработки платежей или рекомендаций выделяется отдельно.
-

◆ Strangler Fig Pattern

- Постепенная миграция: новый функционал реализуется как микросервис, старый монолит постепенно «вымирает».
 - Позволяет уменьшить риски полного переписывания монолита.
-

2. Технические и организационные аспекты при переходе

◆ Технические аспекты

1. **Инфраструктура**
 - Контейнеризация (Docker)
 - Оркестрация (Kubernetes)
 - CI/CD для множества сервисов
2. **Коммуникация**
 - Синхронные (REST/gRPC) и асинхронные (Kafka, RabbitMQ) вызовы
3. **Управление данными**
 - Каждая служба владеет своей БД
 - Переход от ACID-транзакций к Saga и eventual consistency
4. **Наблюдаемость**
 - Логирование, метрики, распределённый трассинг
5. **Безопасность**
 - Единая аутентификация/авторизация (OAuth2, OpenID Connect)

◆ Организационные аспекты

1. **Команды**
 - «Одна команда — один сервис» (автономные команды)
2. **Ownership**
 - Каждая команда отвечает за полный жизненный цикл сервиса
3. **Процесс разработки**
 - Разделение ответственности, стандарты API, контракты
4. **Постепенный переход**
 - Не пытаться переписать весь монолит сразу
 - Использовать Strangler Fig Pattern

3. Паттерны проектирования микросервисов для минимизации проблем с зависимостями и коммуникацией

◆ API Gateway

- Единая точка входа для клиентов
- Сокращает прямые вызовы между сервисами

◆ Circuit Breaker

- Защита от каскадных отказов при сбоях сервисов

◆ Bulkhead

- Изоляция ресурсов для предотвращения распространения проблем между сервисами

◆ Saga

- Управление распределёнными транзакциями через локальные транзакции и компенсации

◆ Event-driven architecture

- Асинхронное взаимодействие через события
- Снижает жёсткую связность и повышает масштабируемость

◆ Shared Library / Contract-first Approach

- Стандартизированные контракты (OpenAPI, protobuf)
- Снижает ошибки и зависимости между командами

1. Что такое многоуровневая архитектура приложений?
2. В чем заключается основная идея паттерна MVC (Model-View-Controller)?
3. Какие преимущества дает использование микросервисной архитектуры перед монолитной?

1. Что такое многоуровневая архитектура приложений

Многоуровневая (или многослойная) архитектура — это структура приложения, разделяющая его на логические слои с разными обязанностями.

Основные уровни:

1. **Presentation Layer (UI)** — отвечает за взаимодействие с пользователем (веб-интерфейс, мобильное приложение).
2. **Business Logic Layer (Service / Domain Layer)** — реализует бизнес-правила и логику.
3. **Data Access Layer (DAL / Repository)** — отвечает за доступ к данным (БД, файловая система, внешние сервисы).

✦ Пример: веб-приложение на Java + Spring: контроллеры → сервисы → репозитории.

Цель: разделение ответственности для лучшей поддерживаемости, тестируемости и масштабируемости.

2. Основная идея паттерна MVC (Model-View-Controller)

MVC — архитектурный паттерн, разделяющий приложение на три компонента:

1. **Model** — данные и бизнес-логика, структура объектов, состояние системы.
2. **View** — визуальное представление данных для пользователя.
3. **Controller** — обрабатывает пользовательский ввод, взаимодействует с Model и обновляет View.

Принцип работы:

- Пользователь совершает действие → Controller обрабатывает → обновляет Model → View отображает изменения.

Преимущества MVC:

- Разделение ответственности → проще тестировать и поддерживать.
 - Позволяет менять UI без изменения бизнес-логики.
 - Упрощает работу команд: дизайнеры работают с View, разработчики — с Model и Controller.
-

3. Преимущества микросервисной архитектуры перед монолитной

Преимущество	Описание
Независимый деплой	Каждый сервис можно разворачивать отдельно без перезапуска всего приложения.
Масштабирование отдельных компонентов	Можно масштабировать только узкие места, а не весь монолит.
Технологическая гибкость	Разные сервисы могут использовать разные языки программирования и БД.
Устойчивость к сбоям	Ошибка одного сервиса не ломает всю систему при правильной изоляции.
Автономные команды	Каждая команда отвечает за свой сервис, ускоряя разработку.
Простота внедрения новых функций	Новые сервисы можно добавлять без сильного влияния на существующие.

✦ Важно: микросервисы решают проблемы масштабируемости и организационной гибкости, но добавляют сложность в коммуникации, мониторинге и транзакциях.

1. Какие основные принципы SOLID применимы к архитектуре приложений?
2. Как реализовать обмен данными между микросервисами?

1. Основные принципы SOLID, применимые к архитектуре приложений

SOLID — это набор принципов проектирования, который повышает поддерживаемость, расширяемость и тестируемость кода. В контексте архитектуры приложений они помогают строить **модули и сервисы с чёткими обязанностями**.

Принцип	Суть	Применение в архитектуре
S — Single Responsibility Principle (SRP)	Класс/сервис должен иметь только одну ответственность	Каждый сервис микросервиса решает одну бизнес-задачу ; легче тестировать и деплоить
O — Open/Closed Principle (OCP)	Модули открыты для расширения, закрыты для модификации	Расширение функциональности через новые сервисы или плагины, не изменяя существующий код
L — Liskov Substitution Principle (LSP)	Подклассы должны быть взаимозаменяемыми с базовыми классами	Интерфейсы сервисов должны позволять менять реализацию без влияния на потребителей
I — Interface Segregation Principle (ISP)	Клиенты не должны зависеть от интерфейсов, которые они не используют	Разделение API микросервисов на узконаправленные интерфейсы
D — Dependency Inversion Principle (DIP)	Зависимости должны строиться на абстракциях, а не на конкретных реализациях	Сервисы зависят от интерфейсов (контрактов), а не от конкретных реализаций других сервисов

✦ В микросервисной архитектуре SOLID помогает строить **независимые, легко расширяемые и заменяемые сервисы**, снижая связность между ними.

2. Реализация обмена данными между микросервисами

Микросервисы — разделённые, автономные процессы, поэтому для обмена данными используют два подхода: **синхронный** и **асинхронный**.

◆ 2.1 Синхронный обмен

- **REST API (HTTP/HTTPS)**
 - JSON или XML
 - Простая интеграция, популярно
 - Пример: один сервис вызывает endpoint другого
- **gRPC**
 - Высокопроизводительные двоичные вызовы
 - Строгая типизация через protobuf
 - Подходит для высоконагруженных систем

Плюсы: простота, прямой ответ

Минусы: блокировка вызова, зависимость от сети

◆ 2.2 Асинхронный обмен

- **Message Brokers / Event-driven**
 - Kafka, RabbitMQ, NATS
 - Сервисы публикуют события и подписываются на них
 - Позволяет строить Saga и eventual consistency
- **Event Sourcing**
 - Каждое событие состояния сохраняется в лог
 - Другие сервисы строят своё состояние из этих событий

Плюсы: высокая устойчивость, масштабируемость, слабая связность

Минусы: сложнее отлаживать, eventual consistency

◆ 2.3 Выбор подхода

Сценарий	Рекомендация
Необходим быстрый ответ / строгая согласованность	Синхронный REST/gRPC
Высокая нагрузка, масштабируемость, отказоустойчивость	Асинхронные события через брокеры
Распределённые транзакции	Event-driven + Saga + Outbox Pattern

1. Опишите процесс проектирования системы с использованием CQRS и Event Sourcing.
2. Какие вызовы и проблемы связаны с переходом от монолитной архитектуры к микросервисной?
3. В чем заключается подход Идемпотентности API и как он влияет на архитектуру системы?

1. Процесс проектирования системы с использованием CQRS и Event Sourcing

◆ Что это такое

- **CQRS (Command Query Responsibility Segregation)** — разделение модели команд (изменение состояния) и запросов (чтение состояния).
 - **Event Sourcing** — состояние системы хранится через **события**, а не через текущие значения данных.
-

◆ Шаги проектирования

1. **Разделение команд и запросов**
 - Определяем действия, которые изменяют состояние (Commands).
 - Определяем запросы, которые возвращают данные (Queries).
 - Создаем отдельные модели данных для чтения и записи.
 2. **Определение событий (Event Sourcing)**
 - Все изменения состояния сохраняются как события.
 - Пример: OrderCreated, PaymentProcessed, InventoryReserved.
 3. **Хранение событий**
 - Используется Event Store (Kafka, EventStoreDB, PostgreSQL/append-only table).
 - История событий становится единственным источником истины.
 4. **Проекция для чтения**
 - События проецируются в read-model для быстрого чтения.
 - Можно создавать несколько проекций под разные запросы.
 5. **Обеспечение консистентности**
 - Асинхронная синхронизация через события.
 - Eventual consistency между read-model и write-model.
-

◆ Преимущества

- Полная история изменений (audit trail).
- Возможность восстановления состояния системы.
- Масштабируемость запросов через отдельные read-model.
- Поддержка сложных бизнес-процессов и Saga.

◆ Недостатки / вызовы

- Повышенная сложность реализации и поддержки.
- Eventual consistency → клиент видит «старые» данные.
- Требуется надежная инфраструктура для событий.

2. Вызовы при переходе от монолита к микросервисной архитектуре

Вызов	Описание
Разделение данных	Каждая служба владеет своей БД → нет глобальных транзакций
Управление зависимостями	Нужны четкие API и контракты между сервисами
Коммуникация между сервисами	Синхронные и асинхронные вызовы, обработка отказов
Мониторинг и трассировка	Логи и метрики распределены → нужен distributed tracing
Организационные аспекты	Команды должны быть автономными, владеть сервисами
Сложность тестирования	Интеграционное тестирование микросервисной системы сложнее

3. Подход идемпотентности API

◆ Что такое идемпотентность

- Идемпотентный API — это API, вызов которого **можно выполнять несколько раз подряд без изменения результата системы после первого вызова.**
- Пример:
 - Метод PUT /order/123 с данными заказа всегда обновит один и тот же заказ без создания дубликатов.
 - Метод POST /payment с уникальным idempotency-key позволяет повторно отправить запрос без повторного списания денег.

◆ Влияние на архитектуру

1. **Обеспечение отказоустойчивости**
 - Повторные вызовы при сбоях не ломают систему.
2. **Асинхронные коммуникации**
 - При работе через очередь сообщений идемпотентность позволяет безопасно повторять обработку события.
3. **Eventual Consistency**
 - Идемпотентные операции упрощают работу с микросервисами и распределенными транзакциями.
4. **Проектирование контрактов**
 - Каждый сервис должен обрабатывать повторы безопасно → меняется логика команд и событий.

1. Что такое DDD (Domain-Driven Design) и для чего он используется?
2. Какие основные компоненты входят в концепцию DDD?
3. Объясните понятие Ubiquitous Language в контексте DDD.

1. Что такое DDD (Domain-Driven Design) и для чего он используется

Domain-Driven Design (DDD) — это подход к разработке программного обеспечения, в котором **фокус смещается на бизнес-домен**.

- Основная идея: **модель предметной области** становится центром архитектуры.
- DDD помогает создавать сложные системы, делая бизнес-логику понятной, согласованной и поддерживаемой.

Для чего используется:

- Для проектирования **сложных, изменяющихся бизнес-доменов**.
- Для обеспечения **единого понимания между разработчиками и бизнесом**.
- Для структурирования кода и сервисов вокруг **реальных бизнес-потребностей**.

✦ Часто применяется вместе с микросервисной архитектурой: каждый сервис реализует свой **Bounded Context** (ограниченный контекст).

2. Основные компоненты концепции DDD

Компонент	Суть
Entity (Сущность)	Объект с уникальной идентичностью, сохраняющейся во времени (например, заказ <code>Order</code>).
Value Object (Объект-значение)	Объект без уникальной идентичности, полностью определяется своими атрибутами (например, <code>Money</code> , <code>Address</code>).
Aggregate (Агрегат)	Группа связанных сущностей и объектов-значений с единой корневой сущностью (<code>Root</code>), через которую управляются изменения.
Repository (Репозиторий)	Абстракция для хранения и получения агрегатов из базы данных.
Service (Сервис домена)	Бизнес-операции, которые не естественно принадлежат ни одной сущности.
Bounded Context (Ограниченный контекст)	Чётко определённая область модели, где термины и правила имеют конкретное значение.
Factory (Фабрика)	Объект или метод для создания сложных агрегатов с соблюдением правил бизнес-домена.

3. Понятие Ubiquitous Language

Ubiquitous Language (Всеобщий язык) — это единый язык, понятный и разработчикам, и бизнесу, который используется во всей модели и коде системы.

- Термины из бизнес-домена напрямую отражаются в коде: классы, методы, события, имена сервисов.
- Помогает:
 - Избегать недопонимания между бизнесом и разработчиками.
 - Сделать код максимально читаемым и отражающим реальную бизнес-логику.

Пример:

- В банковской системе: термин «Account» (счёт) используется одинаково в документации, API, коде и логике событий.

1. Какие преимущества предоставляет использование DDD при проектировании программного обеспечения?
2. В чем разница между Entity и Value Object в DDD?
3. Какова роль агрегатов в DDD и как они помогают управлять сложностью доменной модели?

1. Преимущества использования DDD при проектировании ПО

1. **Сосредоточенность на бизнесе**
 - Модель отражает реальные бизнес-процессы, а не только техническую структуру.
2. **Единое понимание (Ubiquitous Language)**
 - Разработчики и бизнес используют одни и те же термины, что снижает недопонимание.
3. **Управление сложностью**
 - Сложная бизнес-логика разделяется на Bounded Contexts, агрегаты и сервисы.
4. **Поддерживаемость и расширяемость**
 - Логика четко структурирована → проще изменять и добавлять новые функции.
5. **Соответствие микросервисной архитектуре**
 - Каждый Bounded Context можно реализовать как отдельный микросервис.

2. Разница между Entity и Value Object

Параметр	Entity	Value Object
Идентичность	Есть уникальный идентификатор, сохраняющийся во времени	Идентичность не имеет значения; объект определяется своими свойствами
Изменяемость	Обычно изменяемый объект (например, заказ Order)	Обычно неизменяемый; любое изменение → новый объект
Семантика	Концепция, которая существует сама по себе	Концепция, которая имеет смысл только в контексте
Пример	Order, Customer	Money, Address, Email

Вывод: Entity — это «кто», Value Object — это «что» или «характеристика».

3. Роль агрегатов в DDD и как они помогают управлять сложностью

◆ Что такое агрегат

- Агрегат — это группа связанных объектов (Entity + Value Objects), которую рассматривают как единое целое.
- Есть **корневой объект (Aggregate Root)**, через который управляются все операции с агрегатом.

◆ Зачем агрегаты нужны

1. **Инкапсуляция бизнес-правил**
 - Все изменения внутри агрегата проходят через корень → невозможно нарушить целостность данных.
2. **Ограничение области изменений**
 - Внутри агрегата объекты могут взаимодействовать свободно, а снаружи доступ только через корень.
3. **Упрощение транзакций**
 - Только агрегат является единицей атомарности → легче управлять консистентностью и избегать глобальных транзакций.
4. **Управление сложностью**
 - Разделяет систему на независимые агрегаты → меньше связности и лучше масштабируемость.

Пример:

- Агрегат `Order` содержит сущность `OrderItem` и Value Object `Money`.
- Все операции с `OrderItem` происходят через `Order`, корень агрегата.

1. Как DDD интегрируется с микросервисной архитектурой?
2. Обсудите важность и применение контекстных границ (Bounded Contexts) в DDD.
3. Какие стратегии вы бы предложили для реализации DDD в существующей системе с монолитной архитектурой?

1. Интеграция DDD с микросервисной архитектурой

- DDD и микросервисы отлично сочетаются, потому что микросервисы естественно соответствуют **Bounded Context**.
- Каждый микросервис реализует **свой Bounded Context**, владеет собственной моделью предметной области и данными.
- **Преимущества интеграции:**
 1. Сервисы автономны и четко отражают бизнес-домен.
 2. Минимизируются зависимости между сервисами.
 3. Легче управлять сложной бизнес-логикой через агрегаты и доменные события.
 4. Поддерживается Event-Driven подход и Saga для распределенных транзакций.

✦ Пример:

- Order Service — агрегаты Order и OrderItem
- Payment Service — агрегаты Payment и Transaction
- Обмен событиями OrderCreated, PaymentProcessed через брокер сообщений.

2. Важность и применение Bounded Context в DDD

◆ Что такое Bounded Context

- **Bounded Context (BC)** — ограниченный контекст, внутри которого термины и модели имеют строго определённое значение.
- Внутри BC все сущности, агрегаты и Value Objects связаны логикой бизнес-домена.

◆ Почему это важно

1. **Избегает путаницы в терминах**
 - Один и тот же термин может иметь разные значения в разных контекстах.
 - Пример: Customer в CRM ≠ Customer в Billing.
2. **Четкое разграничение ответственности**
 - Каждое BC реализуется отдельно → упрощает разработку и тестирование.
3. **Основа для микросервисов**
 - Обычно один Bounded Context = один микросервис.
4. **Поддержка интеграции через контракты и события**
 - Сервисы взаимодействуют через API или события, не нарушая внутреннюю модель BC.

3. Стратегии реализации DDD в существующей монолитной системе

◆ Стратегия постепенного выделения контекстов

1. **Выделение Bounded Contexts внутри монолита**
 - Разделяем монолит на модули, каждый соответствует ВС.
 - Определяем модели, агрегаты и события для каждого модуля.
2. **Strangler Fig Pattern**
 - Новый функционал реализуется как отдельный микросервис.
 - Старые функции постепенно переписываются и переносятся в ВС/сервис.
3. **Event-Driven интеграция**
 - Внедрение событий для коммуникации между новыми микросервисами и старым монолитом.
 - Позволяет постепенно уйти от глобальных транзакций.
4. **Идемпотентные и контрактно-ориентированные API**
 - Обеспечивает безопасный обмен данными между микросервисами и монолитом.
5. **Анализ зависимостей**
 - Определяем сильные и слабые связи между модулями.
 - Сначала выделяем наиболее автономные и высоконагруженные контексты.